

Étude du protocole ARP

Analyse dans un environnement virtualisé avec pfSense



The graphic features the letters 'ARP' in a large, bold, dark grey font with a thick black outline. Below them, the words 'Address', 'Resolution', and 'Protocol' are stacked vertically in a smaller, orange, cursive-style font with a thin black outline.

ARP
Address
Resolution
Protocol

Auteur :

Date : 25/04/2026

Ulrich Noéjules OGO NDAYEGUIA,

Ingénieur Réseaux & Systèmes/Virtualisation & Cloud

SOMMAIRE

INTRODUCTION.....	1
1. Architecture du réseau.....	2
2. Fonctionnement du protocole ARP.....	6
3. Analyse de la table ARP pfSense	8
4. Analyse Wireshark	10
5. Les vulnérabilités du protocole ARP.....	19
6. Principales attaques exploitant ARP.....	20
7. Mesures de protection	22
Conclusion.....	27

FIGURES

<i>Figure 1 : Architecture réseau.....</i>	<i>2</i>
<i>Figure 2 : Interfaces PfSense.....</i>	<i>2</i>
<i>Figure 3 : Carte réseau physique Windows (NIC)</i>	<i>3</i>
<i>Figure 4 : Interface réseau Kali Linux</i>	<i>4</i>
<i>Figure 5 : Interface réseau vlan Debian.....</i>	<i>4</i>
<i>Figure 6 : Interface réseau vlan Ubuntu</i>	<i>5</i>
<i>Figure 7 : Situation initiale - Absence d'entrée ARP dans le cache</i>	<i>6</i>
<i>Figure 8 : Envoi d'une requête ARP (Broadcast)</i>	<i>6</i>
<i>Figure 9 Réponse ARP du destinataire.....</i>	<i>7</i>
<i>Figure 10 : Mise à jour du cache ARP et envoi des données</i>	<i>7</i>
<i>Figure 11 : Table ARP PfSense</i>	<i>8</i>
<i>Figure 12 : Communication ARP avec Wireshark</i>	<i>10</i>
<i>Figure 13 : Architecture d'adresse MAC</i>	<i>12</i>
<i>Figure 14 : Architecture IPV4</i>	<i>13</i>

TABLEAU

Tableau 1 : Tableau des outils requis 23

INTRODUCTION

Les réseaux informatiques actuels utilisent deux types d'adresses lors de l'interaction entre les appareils connectés : l'adresse IP (adresse logique) et l'adresse MAC (adresse physique). Pour la transmission correcte de données dans le réseau local, il faut mettre en relation ces adresses.

Le protocole ARP (Address Resolution Protocol) est un système qui réalise cette mise en relation de manière dynamique, associant l'adresse IP à l'adresse MAC. Le protocole ARP est toujours utilisé avant le commencement de tout échange d'informations dans le réseau IPv4, ce qui assure correcte transmission de données de niveau lien.

Il faut souligner que le protocole ARP a été élaboré en 1982 dans la RFC 826 par David C. Plummer. À cette époque, dans le contexte des réseaux dits Ethernet, il y avait besoin du développement de l'interface réseau standardisée.

Ainsi, bien que le protocole ARP présente plusieurs insuffisances concernant la protection de l'information, c'est lui qui est toujours utilisé dans les réseaux IPv4, alors que dans les réseaux IPv6 on utilise un autre protocole – NDP (Neighbor Discovery Protocol).

Ce rapport doit consister dans l'analyse théorique et pratique des fonctions du protocole ARP, en utilisant comme base les méthodes de virtualisation (pfSense) et d'analyse du trafic

1. Architecture du réseau local

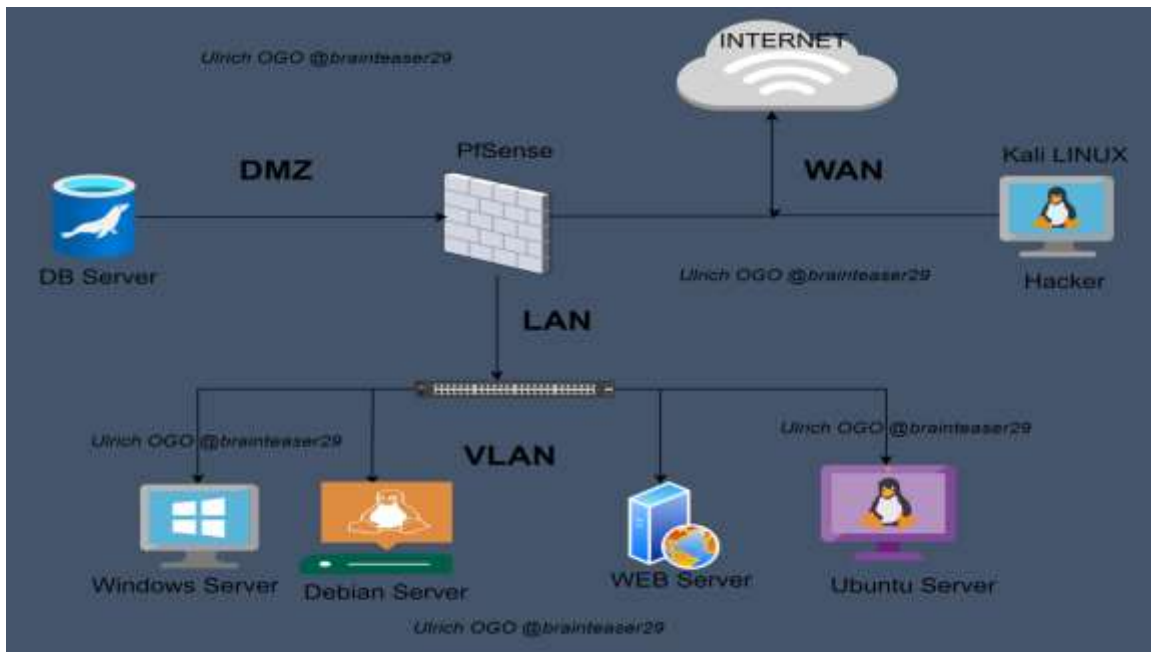


Figure 1 : Architecture réseau

L'environnement est composé de :

- Un firewall/routeur **pfSense**
- Une machine cliente Windows, Debian, Ubuntu et Kali Linux
- Un réseau VLAN (VLAN2 : 10.10.10.0/24)
- Un réseau WAN (192.168.1.0/24)

Interfaces pfSense

Interfaces					
 WAN		1000baseT <full-duplex>	192.168.1.33		
			2001:4278:11:6b03:20c:29ff:fe73:2f27		
 VLAN2		1000baseT <full-duplex>	10.10.10.100		

Figure 2 : Interfaces PfSense

- WAN : 192.168.1.33
- VLAN2 : 10.10.10.100

PfSense joue ici le rôle de :

- Passerelle par défaut
- Serveur DHCP
- Serveur DNS

Machine Windows

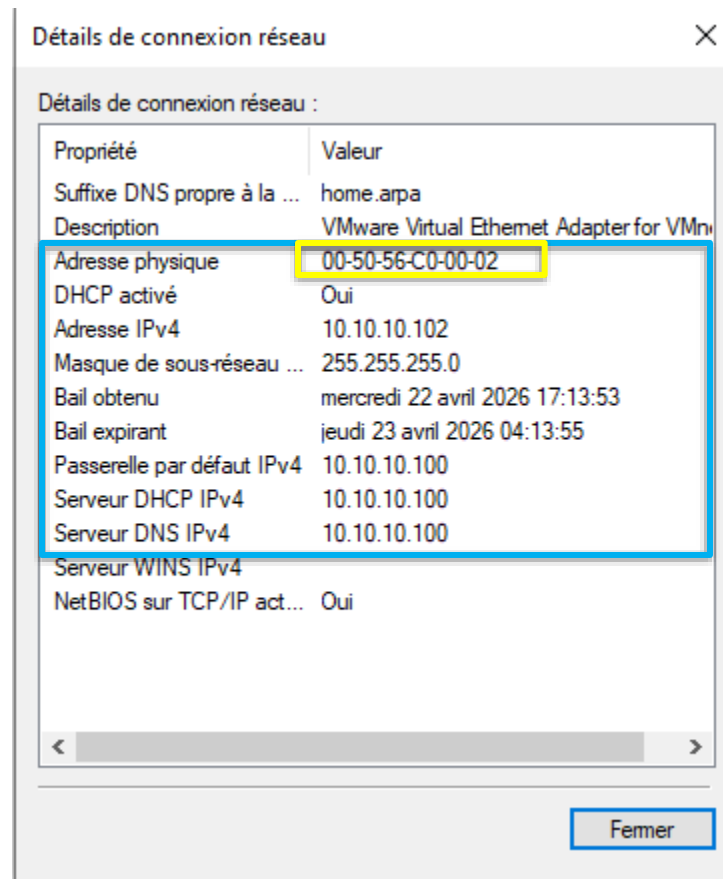


Figure 3 : Carte réseau physique Windows (NIC)

D'après la capture :

- IP : 10.10.10.102
- Masque : 255.255.255.0

- Passerelle : 10.10.10.100
- DHCP activé

Cela montre que la machine dépend de pfSense pour :

- Attribution IP
- Résolution réseau

Machine Kali Linux

```
(tests kali)-[~]
$ ifconfig
eth0: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500
    inet 192.168.1.49 netmask 255.255.255.0 broadcast 192.168.1.255
    inet6 fe80::20c:29ff:fedd:783e prefixlen 64 scopeid 0x20<link>
    inet6 2001:4278:11:6b03:20c:29ff:fedd:783e prefixlen 64 scopeid 0x0<global>
    ether 00:0c:29:dd:78:3e txqueuelen 1000 (Ethernet)
    RX packets 445 bytes 39876 (38.9 KiB)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 912 bytes 65713 (64.1 KiB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

lo: flags=73<UP,LOOPBACK,RUNNING> mtu 65536
    inet 127.0.0.1 netmask 255.0.0.0
    inet6 ::1 prefixlen 128 scopeid 0x10<host>
    loop txqueuelen 1000 (Boucle locale)
    RX packets 484 bytes 45980 (44.9 KiB)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 484 bytes 45980 (44.9 KiB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0
```

Figure 4 : Interface réseau Kali Linux

- IP : 192.168.1.49
- Interface : eth0

Machine Debian

```
ens36: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500
    inet6 fe80::20c:29ff:feaa:fb72 prefixlen 64 scopeid 0x20<link>
    ether 00:0c:29:aa:fb:72 txqueuelen 1000 (Ethernet)
    RX packets 768 bytes 69233 (67.6 KiB)
    RX errors 0 dropped 6 overruns 0 frame 0
    TX packets 18609 bytes 1135720 (1.0 MiB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

ens36.2: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500
    inet 10.10.10.2 netmask 255.255.255.0 broadcast 10.10.10.255
    inet6 fe80::20c:29ff:feaa:fb72 prefixlen 64 scopeid 0x20<link>
    ether 00:0c:29:aa:fb:72 txqueuelen 1000 (Ethernet)
    RX packets 762 bytes 57641 (56.2 KiB)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 18585 bytes 805726 (786.8 KiB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0
```

Figure 5 : Interface réseau vlan Debian

- Interface VLAN (ens36.2)
- IP : 10.10.10.2

Machine Ubuntu

```
vlan2: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500
    inet 10.10.10.3 netmask 255.255.255.0 broadcast 10.10.10.255
    inet6 fe80::20c:29ff:fe4f:958a prefixlen 64 scopeid 0x20<link>
    ether 00:0c:29:4f:95:8a txqueuelen 1000 (Ethernet)
    RX packets 78 bytes 3775 (3.7 KB)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 258 bytes 11364 (11.3 KB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0
```

Figure 6 : Interface réseau vlan Ubuntu

- Interface VLAN (ens36.2)
- IP : 10.10.10.3

Le réseau étudié comprend un firewall pfSense, une machine Windows, une machine Debian, une machine Ubuntu et une machine Kali. Le réseau VLAN2 (10.10.10.0/24) est utilisé pour la communication interne.

2. Fonctionnement du protocole ARP

Lorsqu'une machine souhaite communiquer avec une autre, elle doit connaître son adresse MAC. Si elle ne la connaît pas, elle envoie une requête ARP en **broadcast** et en retour elle obtient une réponse ARP en **unicast**.

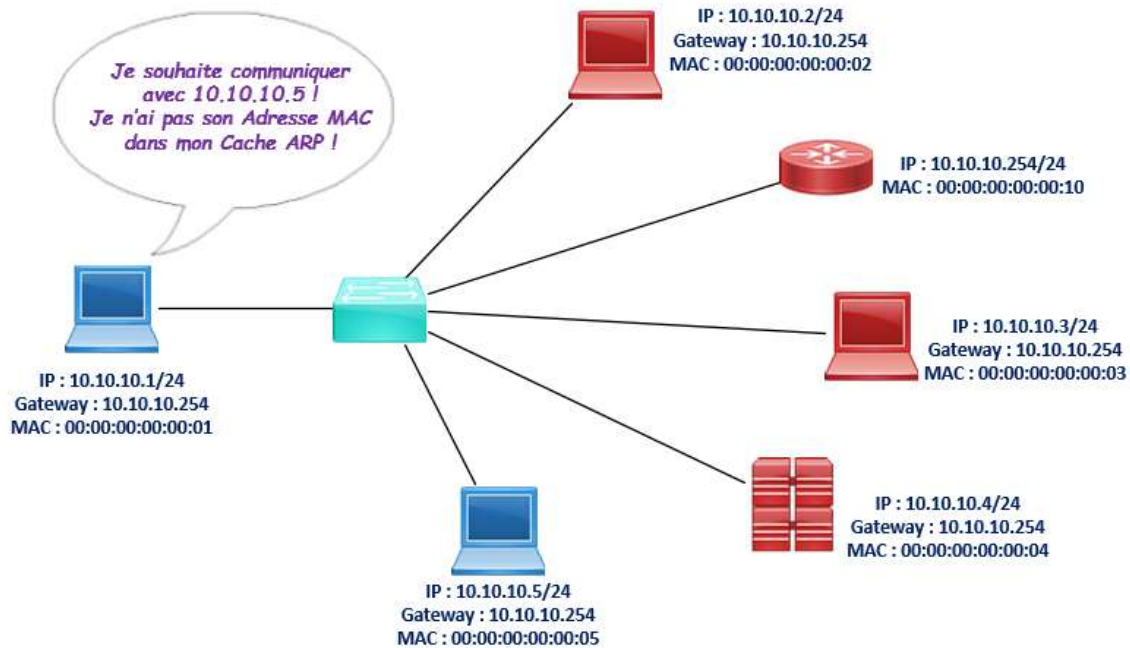


Figure 7 : Situation initiale - Absence d'entrée ARP dans le cache

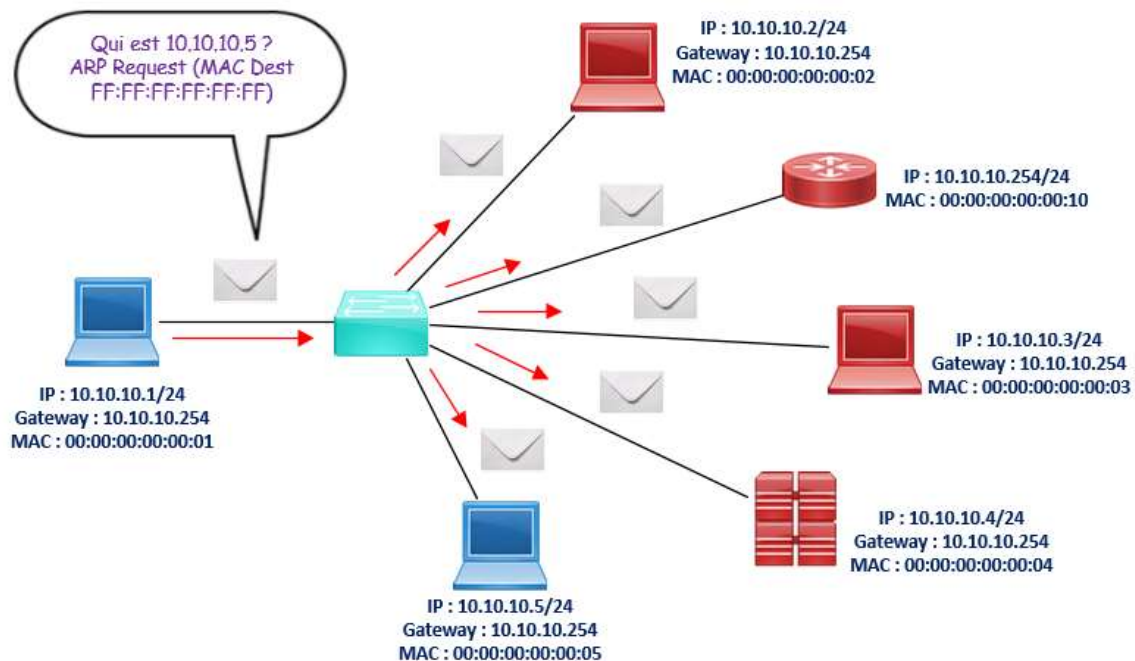


Figure 8 : Envoi d'une requête ARP (Broadcast)

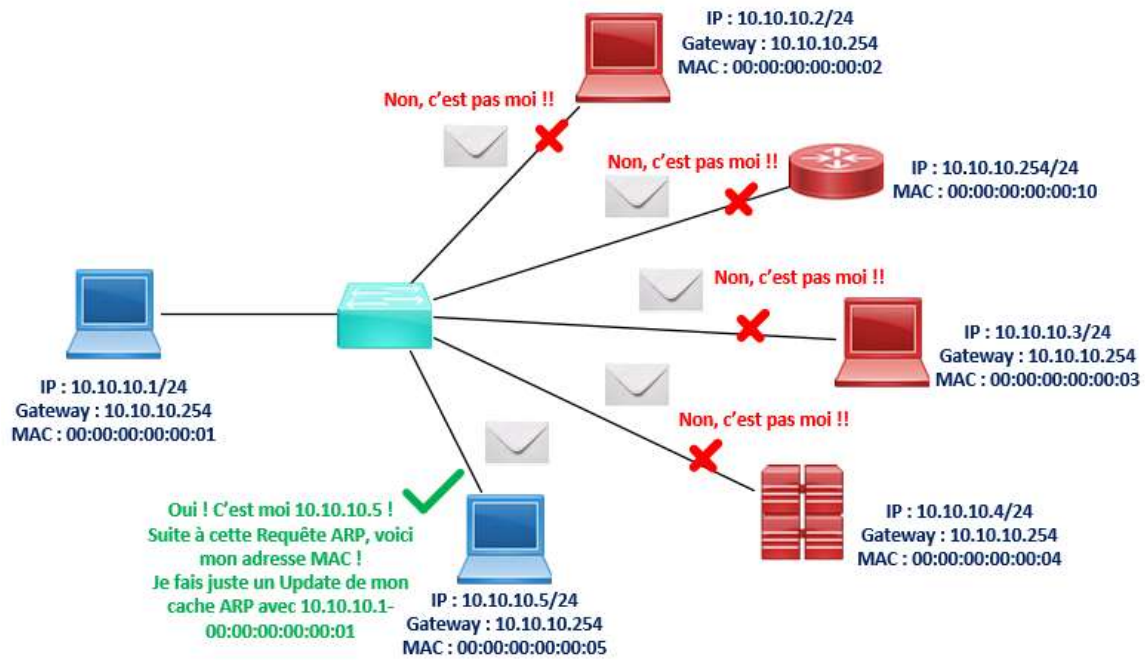


Figure 9 Réponse ARP du destinataire

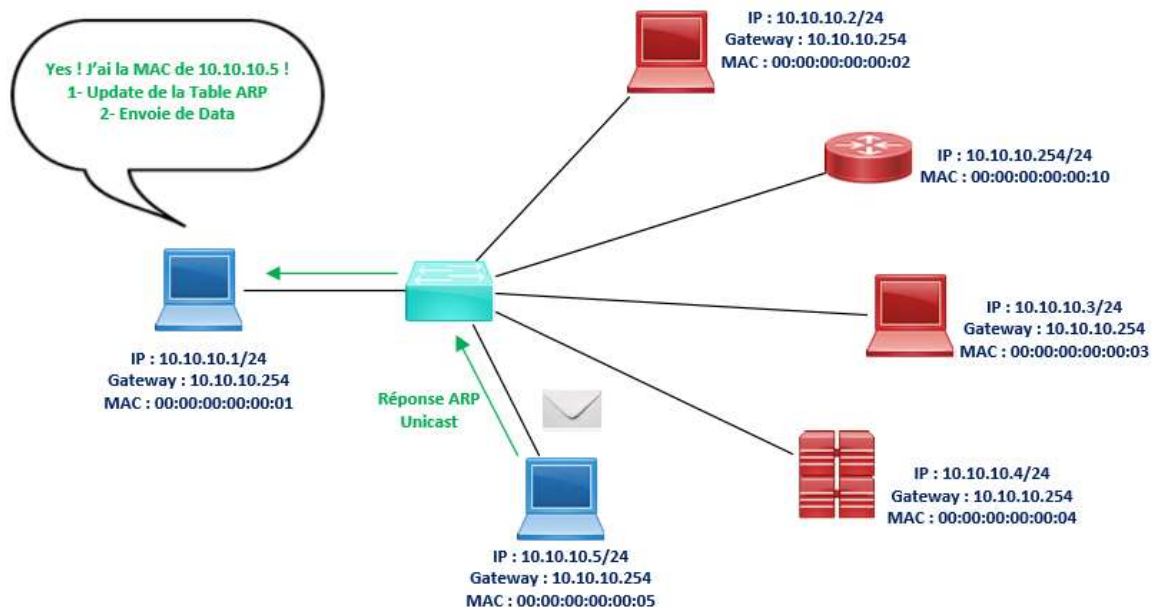


Figure 10 : Mise à jour du cache ARP et envoi des données

Cas réel : communication dans VLAN2

Quand la machine Windows (10.10.10.102) veut communiquer avec Debian (10.10.10.2) :

Étapes :

1. Vérification du cache ARP
2. Si absence → envoi ARP Request (broadcast) :

Who has 10.10.10.2 ?

Tell 10.10.10.102

3. Réponse de Debian (ARP Reply) :

10.10.10.2 is at 00:0c:29:aa:fb:72

4. Mise à jour du cache ARP
5. Communication directe

3. Analyse de la table ARP pfSense

ARP Table						
Interface	IP address	MAC address	Hostname	Status	Link Type	Actions
VLAN2	10.10.10.2	00:0c:29:aa:fb:72		Expires in 1162 seconds	vlan	
VLAN2	10.10.10.3	00:0c:29:4f:95:8a		Expires in 1179 seconds	vlan	
WAN	192.168.1.41	00:0c:29:73:2f:27		Permanent	ethernet	
WAN	192.168.1.14	a4:02:b9:07:1a:b6		Expires in 1109 seconds	ethernet	
WAN	192.168.1.1	bc:bd:84:a7:93:01		Expires in 1190 seconds	ethernet	
VLAN2	10.10.10.102	00:50:56:c0:00:02	DESKTOP-U99MHUL	Expires in 1142 seconds	vlan	
VLAN2	10.10.10.100	00:0c:29:73:2f:31	pfSense.home.arp	Permanent	vlan	

Figure 11 : Table ARP PfSense

La capture pfSense montre plusieurs entrées ARP :

Exemple :

- 10.10.10.2 → 00:0c:29:aa:fb:72
- 10.10.10.102 → 00:50:56:c0:00:02

Cela confirme que :

- PfSense apprend dynamiquement les correspondances IP/MAC
- Chaque machine du VLAN est connue via ARP

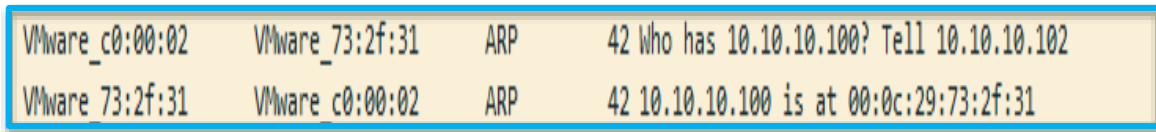
Types d'entrées :

- **Dynamic (Expires)** → apprises automatiquement
- **Permanent** → statiques (ex : pfSense lui-même)

4. Analyse Wireshark

4.1 Contexte

Une capture réseau a été réalisée avec Wireshark afin d'observer les échanges ARP entre une machine cliente et la passerelle pfSense.



VMware_c0:00:02	VMware_73:2f:31	ARP	42 Who has 10.10.10.100? Tell 10.10.10.102
VMware_73:2f:31	VMware_c0:00:02	ARP	42 10.10.10.100 is at 00:0c:29:73:2f:31

Figure 12 : Communication ARP avec Wireshark

4.2 ARP Request

Analyse :

- **Type** : ARP Request
- **Source MAC** : 00:50:56:c0:00:02
- **Destination MAC** : ff:ff:ff:ff:ff:ff (broadcast)
- **IP source** : 10.10.10.102
- **IP cible** : 10.10.10.100

Who has 10.10.10.100? Tell 10.10.10.102

La machine cliente cherche l'adresse MAC de la passerelle.

Explication :

La machine 10.10.10.102 cherche à connaître l'adresse MAC de la passerelle 10.10.10.100 (pfSense).

Elle envoie donc une requête ARP en **broadcast** à tout le réseau :

“Qui possède 10.10.10.100 ?”

4.3 ARP Reply

Analyse :

- **Type** : ARP Reply
- **Source MAC** : 00:0c:29:73:2f:31 (pfSense)
- **Destination MAC** : 00:50:56:c0:00:02
- **IP source** : 10.10.10.100
- **IP destination** : 10.10.10.102

10.10.10.100 is at 00:0c:29:73:2f:31

pfSense répond avec son adresse MAC.

Explication :

Le firewall **pfSense** répond directement à la machine cliente :

“Je suis 10.10.10.100, voici mon adresse MAC” Cette réponse est envoyée en **unicast**.

4.4 Analyse des champs

Les champs importants incluent : opcode, adresse MAC source, adresse IP source, adresse MAC cible et adresse IP cible.

Chaque trame ARP contient des champs essentiels :

Opcode (Operation Code) : est un champ d’une trame ARP qui indique **le type de message ARP envoyé**. Ex : **ARP Request** (requête) et **ARP Reply** (réponse). L’opcode permet aux machines de comprendre immédiatement : “Est-ce que c’est une question ou une réponse ?”

Opcode

- **1** → ARP Request
- **2** → ARP Reply

Dans notre capture :

- Ligne 1 → Opcode = 1
- Ligne 2 → Opcode = 2

L'**adresse MAC (Media Access Control)** est une adresse physique unique attribuée à une carte réseau qui a une longueur de **48 bits (6 octets)** et un format **hexadécimal**. Elle a pour rôle d'identifier un équipement de manière unique dans un réseau local (LAN) et fonctionne au niveau **couche 2 (liaison)** utilisée par **Ethernet**. C'est grâce à elle que les données sont envoyées au bon appareil physiquement.



Figure 13 : Architecture d'adresse MAC

Adresses MAC

- **Sender MAC** : machine qui envoie la trame
- **Target MAC** :
 - Inconnue dans la requête (00:00:00:00:00:00)
 - Connue dans la réponse

L'**adresse IP** est une **adresse logique** attribuée à un équipement dans un réseau qui a pour format IPV4 de longueur 32 bits. Elle a pour **rôle** d'identifier une machine dans un réseau **et** de router les paquets entre différents réseaux. C'est une **adresse logique**, contrairement à la MAC. Elle fonctionne au niveau **couche 3 (réseau)** et peut changer (**DHCP**).

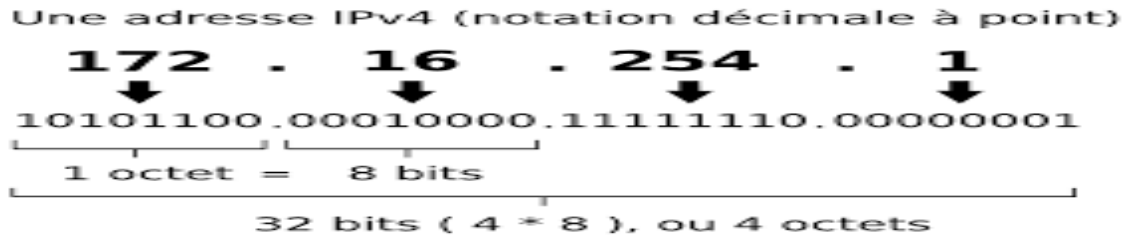


Figure 14 : Architecture IPV4

Adresses IP

- **Sender IP** : IP de l'émetteur
- **Target IP** : IP recherchée

Broadcast vs Unicast

- ARP Request → **Broadcast**

ff:ff:ff:ff:ff:ff

- ARP Reply → **Unicast**

Cela optimise le réseau en évitant les réponses inutiles.

4.5 Cas particulier : communication inter-réseaux

Quand une machine du VLAN2 veut contacter une machine WAN :

Elle ne cherche PAS la MAC de la destination

Elle cherche la MAC de la passerelle (pfSense) → 00:0c:29:73:2f:31

Exemple :

Destination : 192.168.1.49

ARP cible : 10.10.10.100 (pfSense)

4.6 Problème rencontré

Les machines du réseau ne recevaient pas les requêtes ARP, cela fait qu'il y avait un problème de communication, c'est-à-dire les machines du réseau n'arrivaient pas à acheminer correctement les trames Ethernet au niveau couche 2 (Layer 2) du moins contacter la passerelle **PfSense**.

Tentative d'obtention d'adresse MAC PfSense côté Debian → Echec

```
Linux dc 5.10.0-35-amd64 #1 SMP Debian 5.10.237-1 (2025-05-19) x86_64
```

```
The programs included with the Debian GNU/Linux system are free software;  
the exact distribution terms for each program are described in the  
individual files in /usr/share/doc/*/copyright.
```

```
Debian GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent  
permitted by applicable law.
```

```
***** Brainteaser29 baby *****
```

```
You're welcome !!!
```

```
Last login: Sat Apr 25 14:37:16 2026 from 10.10.10.102
```

```
root@dc:~# ip neigh
```

```
10.10.10.100 dev ens36.2 FAILED
```

```
10.10.10.102 dev ens36.2 lladdr 00:50:56:c0:00:02 REACHABLE
```

```
root@dc:~# arp -a
```

```
? (10.10.10.100) at <incomplete> on ens36.2
```

```
? (10.10.10.102) at 00:50:56:c0:00:02 [ether] on ens36.2
```

```
root@dc:~#
```

Tentative d'obtention d'adresse MAC PfSense côté Ubuntu → Echec

```
root@server:~# ip neigh
```

```
10.10.10.100 dev vlan2 INCOMPLETE
```

```
10.10.10.102 dev vlan2 lladdr 00:50:56:c0:00:02 REACHABLE
```

```
root@server:~# arp -a
```

```
? (10.10.10.100) at <incomplete> on vlan2
```

```
? (10.10.10.102) at 00:50:56:c0:00:02 [ether] on vlan2
```

```
root@server:~#
```

Problème de joignabilité sur le réseau

Il y a aussi ce problème de réponses aux requêtes ping, quand la machine Debian envoie un message « Echo Request » (ping) vers la machine Ubuntu → **Hôte de destination Injoignable !!!**

```
root@dc:~# ping 10.10.10.3
PING 10.10.10.3 (10.10.10.3) 56(84) bytes of data.
From 10.10.10.2 icmp_seq=1 Destination Host Unreachable
From 10.10.10.2 icmp_seq=2 Destination Host Unreachable
From 10.10.10.2 icmp_seq=3 Destination Host Unreachable
^C
--- 10.10.10.3 ping statistics ---
5 packets transmitted, 0 received, +3 errors, 100% packet loss, time 4079ms
pipe 4
root@dc:~# █
```

Vérifions maintenant la cache ARP de Debian → Echec, on ne parvient pas à récupérer l'adresse MAC de la machine Ubuntu (**FAILED**).

```
root@dc:~# ip neigh
10.10.10.100 dev ens36.2 lladdr 00:0c:29:73:2f:31 REACHABLE
10.10.10.3 dev ens36.2 FAILED
10.10.10.102 dev ens36.2 lladdr 00:50:56:c0:00:02 REACHABLE
root@dc:~#
root@dc:~# arp -a
? (10.10.10.100) at 00:0c:29:73:2f:31 [ether] on ens36.2
? (10.10.10.3) at <incomplete> on ens36.2
? (10.10.10.102) at 00:50:56:c0:00:02 [ether] on ens36.2
root@dc:~# _
```

Pareil, un message « Echo Request » (ping) de la machine Ubuntu vers la machine Debian → **Hôte de destination Injoignable !!!**

```
root@server:~# ping 10.10.10.2
PING 10.10.10.2 (10.10.10.2) 56(84) bytes of data.
From 10.10.10.3 icmp_seq=1 Destination Host Unreachable
From 10.10.10.3 icmp_seq=2 Destination Host Unreachable
From 10.10.10.3 icmp_seq=3 Destination Host Unreachable
^C
--- 10.10.10.2 ping statistics ---
5 packets transmitted, 0 received, +3 errors, 100% packet loss, time 4101ms
pipe 3
root@server:~# _
```

Vérifions la cache ARP du côté Ubuntu → Echec, on ne parvient pas à récupérer l'adresse MAC de la machine Debian (**FAILED**).

```
root@server:~# ip neigh
10.10.10.102 dev vlan2 lladdr 00:50:56:c0:00:02 STALE
10.10.10.100 dev vlan2 lladdr 00:0c:29:73:2f:31 REACHABLE
10.10.10.2 dev vlan2 FAILED
root@server:~#
root@server:~# arp -a
? (10.10.10.102) at 00:50:56:c0:00:02 [ether] on vlan2
? (10.10.10.100) at 00:0c:29:73:2f:31 [ether] on vlan2
? (10.10.10.2) at <incomplete> on vlan2
root@server:~#
```

Cause probable :

- Mauvaise configuration VLAN
- Problème de joignabilité
- Interface mal taggée / access
- Mauvais management de vlan sur la carte réseau physique (NIC)

Solution appliquée :

- Correction de la configuration réseau
- Redémarrage du pare-feu **PfSense**
- Désactiver IPv6 sur la carte réseau physique
- Désactivation et réactivation de la carte réseau physique (NIC)
- Attribution DHCP dynamique via le router/pare-feu **PfSense**

Résultat obtenu :

ARP fonctionne correctement côté machine Debian

Linux dc 5.10.0-35-amd64 #1 SMP Debian 5.10.237-1 (2025-05-19) x86_64

The programs included with the Debian GNU/Linux system are free software; the exact distribution terms for each program are described in the individual files in /usr/share/doc/*/copyright.

Debian GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent permitted by applicable law.

***** Brainteaser29 baby *****

You're welcome !!!

Last login: Sat Apr 25 14:36:58 2026 from 10.10.10.102

root@dc:~#

root@dc:~# ip neigh

10.10.10.100 dev ens36.2 lladdr 00:0c:29:73:2f:31 REACHABLE

10.10.10.102 dev ens36.2 lladdr 00:50:56:c0:00:02 REACHABLE

root@dc:~#

root@dc:~# arp -a

? (10.10.10.100) at 00:0c:29:73:2f:31 [ether] on ens36.2

? (10.10.10.102) at 00:50:56:c0:00:02 [ether] on ens36.2

root@dc:~#

ARP fonctionne correctement côté machine Ubuntu

root@server:~# ip neigh

10.10.10.102 dev vlan2 lladdr 00:50:56:c0:00:02 REACHABLE

10.10.10.100 dev vlan2 lladdr 00:0c:29:73:2f:31 REACHABLE

root@server:~#

root@server:~# arp -a

? (10.10.10.102) at 00:50:56:c0:00:02 [ether] on vlan2

? (10.10.10.100) at 00:0c:29:73:2f:31 [ether] on vlan2

root@server:~#

Les machines communiquent : Debian → Ubuntu

```
root@dc:~# ping 10.10.10.3
PING 10.10.10.3 (10.10.10.3) 56(84) bytes of data.
64 bytes from 10.10.10.3: icmp_seq=1 ttl=64 time=0.435 ms
64 bytes from 10.10.10.3: icmp_seq=2 ttl=64 time=0.574 ms
^C
--- 10.10.10.3 ping statistics ---
2 packets transmitted, 2 received, 0% packet loss, time 1001ms
rtt min/avg/max/mdev = 0.435/0.504/0.574/0.069 ms
root@dc:~# █
```

```
root@dc:~# ping 10.10.10.3
PING 10.10.10.3 (10.10.10.3) 56(84) bytes of data.
64 bytes from 10.10.10.3: icmp_seq=1 ttl=64 time=0.433 ms
64 bytes from 10.10.10.3: icmp_seq=2 ttl=64 time=0.960 ms
^C
--- 10.10.10.3 ping statistics ---
2 packets transmitted, 2 received, 0% packet loss, time 1001ms
rtt min/avg/max/mdev = 0.433/0.696/0.960/0.263 ms
root@dc:~# ip neigh
10.10.10.3 dev ens36.2 lladdr 00:0c:29:4f:95:8a DELAY
10.10.10.100 dev ens36.2 lladdr 00:0c:29:73:2f:31 REACHABLE
10.10.10.102 dev ens36.2 lladdr 00:50:56:c0:00:02 REACHABLE
root@dc:~# arp -a
? (10.10.10.3) at 00:0c:29:4f:95:8a [ether] on ens36.2
? (10.10.10.100) at 00:0c:29:73:2f:31 [ether] on ens36.2
? (10.10.10.102) at 00:50:56:c0:00:02 [ether] on ens36.2
root@dc:~#
```

Les machines communiquent : Ubuntu → Debian

```
root@server:~# ping 10.10.10.2
PING 10.10.10.2 (10.10.10.2) 56(84) bytes of data.
64 bytes from 10.10.10.2: icmp_seq=1 ttl=64 time=0.421 ms
64 bytes from 10.10.10.2: icmp_seq=2 ttl=64 time=0.912 ms
^C
--- 10.10.10.2 ping statistics ---
2 packets transmitted, 2 received, 0% packet loss, time 1007ms
rtt min/avg/max/mdev = 0.421/0.666/0.912/0.245 ms
```

```
root@server:~# ping 10.10.10.2
PING 10.10.10.2 (10.10.10.2) 56(84) bytes of data.
64 bytes from 10.10.10.2: icmp_seq=1 ttl=64 time=0.421 ms
64 bytes from 10.10.10.2: icmp_seq=2 ttl=64 time=0.912 ms
^C
--- 10.10.10.2 ping statistics ---
2 packets transmitted, 2 received, 0% packet loss, time 1007ms
rtt min/avg/max/mdev = 0.421/0.666/0.912/0.245 ms
root@server:~# ip neigh
10.10.10.102 dev vlan2 lladdr 00:50:56:c0:00:02 REACHABLE
10.10.10.2 dev vlan2 lladdr 00:0c:29:aa:fb:72 REACHABLE
10.10.10.100 dev vlan2 lladdr 00:0c:29:73:2f:31 REACHABLE
root@server:~# arp -a
? (10.10.10.102) at 00:50:56:c0:00:02 [ether] on vlan2
? (10.10.10.2) at 00:0c:29:aa:fb:72 [ether] on vlan2
? (10.10.10.100) at 00:0c:29:73:2f:31 [ether] on vlan2
root@server:~#
```

5. Les vulnérabilités du protocole ARP

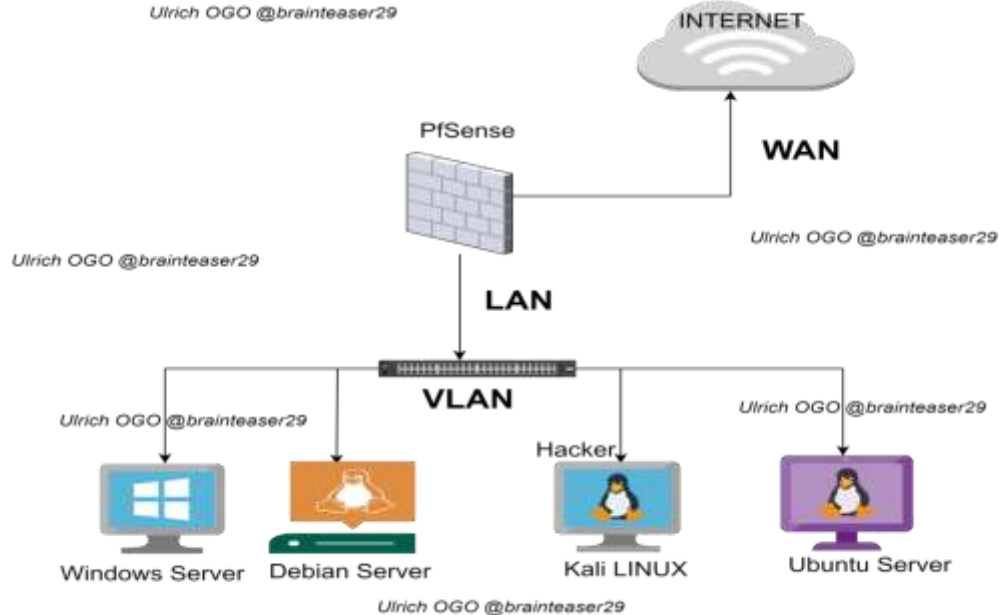


Figure 15 : Architecture Man-In-The-Middle

5.1 Contexte

ARP (Address Resolution Protocol), qui fait partie intégrante des LANs (réseaux locaux), sert à associer l'adresse IP avec l'adresse MAC. Bien que le protocole ARP joue un rôle très important dans les LANs, il possède de nombreuses faiblesses en termes de sécurité, essentiellement en raison de l'absence de moyens pour l'authentifier et de la vérifier.

Ces faiblesses sont largement utilisées lors d'attaques par des hackers dans des LANs. C'est ce qui montre clairement les failles et faiblesses du protocole ARP.

5.2 Absence d'authentification

ARP ne vérifie pas l'identité de l'émetteur d'une requête ou d'une réponse.

Conséquence :

- N'importe quel hôte peut envoyer une réponse ARP falsifiée
- Les machines acceptent les réponses sans vérification

5.3 Mise à jour automatique du cache ARP

Les systèmes mettent automatiquement à jour leur cache ARP dès réception d'une réponse ARP, même si aucune requête n'a été envoyée.

Conséquence :

- Possibilité d'injecter de fausses entrées ARP
- Facilite les attaques de type spoofing

5.4 Communication en clair (non chiffrée)

ARP ne chiffre aucune donnée.

Conséquence :

- Les informations peuvent être interceptées facilement
- Permet l'écoute du trafic réseau (sniffing)

5.5 Stateless protocol (sans état)

ARP ne garde pas trace des requêtes envoyées.

Conséquence :

- Une réponse ARP peut être acceptée même si aucune requête correspondante n'a été émise

6. Principales attaques exploitant ARP

6.1 ARP Spoofing (ou ARP Poisoning)

Définition

Technique consistant à envoyer de fausses réponses ARP pour associer l'adresse IP d'une victime à l'adresse MAC de l'attaquant.

Objectif

Son objectif est d'intercepter le trafic, se positionner en Man-In-The-Middle (MITM).

Fonctionnement

1. Quand l'attaquant envoie une réponse ARP falsifiée : « L'adresse IP de la passerelle = mon adresse MAC »
2. La victime met à jour son cache ARP
3. Le trafic passe désormais par l'attaquant

6.2 Attaque Man-In-The-Middle (MITM)

Définition

L'attaquant s'interpose entre deux machines pour intercepter et éventuellement modifier les communications.

Impact

- Vol de données sensibles (mots de passe, cookies)
- Modification du trafic
- Injection de contenu malveillant

6.3 ARP Flooding

Définition

Envoi massif de requêtes ou réponses ARP pour saturer le réseau ou la table ARP.

Impact

- Dégradation des performances réseau
- Dénî de service (DoS)

6.4 Session Hijacking

Définition

Détournement d'une session active grâce à l'interception du trafic via ARP spoofing.

Impact

- Prise de contrôle de sessions utilisateur
- Accès non autorisé

7. Mesures de protection

7.1 Entrées ARP statiques

Configurer manuellement les correspondances IP ↔ MAC.

- Avantage : sécurisé
- Inconvénient : difficile à gérer à grande échelle

7.2 Utilisation de VLANs

Segmenter et isoler le réseau pour limiter la portée des attaques.

7.3 Sécurisation via protocoles supérieurs

- HTTPS
- SSH
- VPN

Et ce n'est pas tout, ces protocoles protègent les données même en cas d'attaque MITM étant donné que tout le trafic est chiffré en d'autres termes, ce sont des protocoles sécurisés.

7.4 Exploitation des vulnérabilités ARP avec Kali Linux

Objectif du TP

Mettre en œuvre une attaque :

- ARP Spoofing
- Man-In-The-Middle (MITM)
- Capture de trafic (sniffing)

1. Architecture du lab

Machines nécessaires

Machine	Rôle	Adresse IP
Kali Linux	Attaquant	10.10.10.4
Victime	Client Windows	10.10.10.102
Routeur/PfSense	Passerelle	10.10.10.100

Tableau 1 : Tableau des outils requis

2. Préparation de l'environnement

Activer le forwarding IP sur Kali :

Pourquoi activer le forwarding IP ?

- ✓ C'est pour transmettre les paquets entre victime et passerelle
- ✓ Éviter de couper la connexion (sinon DoS).

```
echo 1 > /proc/sys/net/ipv4/ip_forward
```

Vérifions le réseau

```
ip a
```

Listons les ARP

```
arp -a
```


L'attaque MITM est 100% fonctionnel :

- ✓ ARP intercepté
- ✓ DNS intercepté
- ✓ Trafic victime visible

La preuve, la victime **10.10.10.102** demande :

- IP de gator.volces.com
- IP de chat.deepseek.com

```
(root@kali)-[~]
# tcpdump -i eth0.2 port 53
tcpdump: verbose output suppressed, use -v[v]... for full protocol decode
listening on eth0.2, link-type EN10MB (Ethernet), snapshot length 262144 bytes
09:44:06.547456 IP 10.10.10.102.61610 > 10.10.10.100.domain: 28724+ A? clientservices.googleapis.com. (47)
09:44:11.899351 IP 10.10.10.102.64814 > 10.10.10.100.domain: 11435+ A? ntp.msn.com. (29)
09:44:12.900639 IP 10.10.10.102.64814 > 10.10.10.100.domain: 11435+ A? ntp.msn.com. (29)
09:44:14.910093 IP 10.10.10.102.64814 > 10.10.10.100.domain: 11435+ A? ntp.msn.com. (29)
09:44:18.922117 IP 10.10.10.102.64814 > 10.10.10.100.domain: 11435+ A? ntp.msn.com. (29)
09:44:23.100311 IP 10.10.10.102.54014 > 10.10.10.100.domain: 18058+ A? clientservices.googleapis.com. (47)
09:44:36.088411 IP 10.10.10.102.51457 > 10.10.10.100.domain: 26157+ A? functional.events.data.microsoft.com. (54)
09:44:37.091588 IP 10.10.10.102.51457 > 10.10.10.100.domain: 26157+ A? functional.events.data.microsoft.com. (54)
09:44:39.099477 IP 10.10.10.102.51457 > 10.10.10.100.domain: 26157+ A? functional.events.data.microsoft.com. (54)
09:44:42.092932 IP 10.10.10.102.61845 > 10.10.10.100.domain: 4893+ A? functional.events.data.microsoft.com. (54)
09:44:48.328238 IP 10.10.10.102.64123 > 10.10.10.100.domain: 33010+ A? chat.deepseek.com. (35)
09:44:49.329496 IP 10.10.10.102.64123 > 10.10.10.100.domain: 33010+ A? chat.deepseek.com. (35)
09:44:51.262322 IP 10.10.10.102.63922 > 10.10.10.100.domain: 61185+ A? gator.volces.com. (34)
09:44:51.921914 IP 10.10.10.102.49392 > 10.10.10.100.domain: 63732+ A? ntp.msn.com. (29)
09:45:01.903841 IP 10.10.10.102.60788 > 10.10.10.100.domain: 65512+ A? ntp.msn.com. (29)
09:45:01.919208 IP 10.10.10.102.51383 > 10.10.10.100.domain: 32413+ A? www.bing.com. (30)
09:45:02.182425 IP 10.10.10.102.49458 > 10.10.10.100.domain: 8651+ A? th.bing.com. (29)
09:45:02.182427 IP 10.10.10.102.54696 > 10.10.10.100.domain: 53406+ A? img-s-msn-com.akamaized.net. (45)
09:45:02.290631 IP 10.10.10.102.54883 > 10.10.10.100.domain: 8001+ A? c.msn.com. (27)
09:45:02.352496 IP 10.10.10.102.58471 > 10.10.10.100.domain: 20533+ A? c.bing.com. (28)
09:45:02.367999 IP 10.10.10.102.55604 > 10.10.10.100.domain: 42160+ A? assets.msn.com. (32)
09:45:02.600250 IP 10.10.10.102.60883 > 10.10.10.100.domain: 15202+ A? api.msn.com. (29)
09:45:03.296224 IP 10.10.10.102.55402 > 10.10.10.100.domain: 43554+ A? srtb.msn.com. (30)
09:45:08.788760 IP 10.10.10.102.55428 > 10.10.10.100.domain: 19177+ A? clients4.google.com. (37)
09:45:14.744574 IP 10.10.10.102.60821 > 10.10.10.100.domain: 13443+ A? clients4.google.com. (37)
09:45:21.975692 IP 10.10.10.102.58350 > 10.10.10.100.domain: 45178+ A? ntp.msn.com. (29)
09:45:31.974868 IP 10.10.10.102.52105 > 10.10.10.100.domain: 44984+ A? ntp.msn.com. (29)
^C
27 packets captured
27 packets received by filter
0 packets dropped by kernel
```

Capture de paquet au **port 53** (DNS) avec tcpdump. Je suis présentement connecté aux deux navigateurs sur mon client Windows : **Microsoft** et **Google** y compris plusieurs sites visités. La preuve, tout le trafic est retracé. Cela facilitera la tâche à l'attaquant (Hacker) de vite m'atteindre (piéger) étant donné que ce dernier connaît déjà mes traces etc. Quand les infos non chiffrées arrivent, il les intercepte et les manipule.

Victime (10.10.10.102) ↔ Hacker (Ulrich) ↔ Gateway/DNS (10.10.10.100)



Conclusion

Pour en finir, le protocole ARP est essentiel pour la communication réseau en IPv4. Cette étude a permis de comprendre son fonctionnement à travers une analyse pratique. Sans oublier les sites auxquels je me suis connecté, c'est exactement mon trafic en ligne qui a été retracé. Les analyses et les résultats ont prouvé qu'à quel point l'attaque Man-In-The-Middle (MITM) est fatale et peut causer des dégâts dans les LANs.

Table des matières

SOMMAIRE	I
FIGURES.....	II
TABLEAU.....	III
INTRODUCTION.....	1
1. Architecture du réseau local.....	2
2. Fonctionnement du protocole ARP.....	6
3. Analyse de la table ARP pfSense	8
4. Analyse Wireshark	10
4.1 Contexte	10
4.2 ARP Request.....	10
4.3 ARP Reply.....	11
4.4 Analyse des champs	11
4.5 Cas particulier : communication inter-réseaux	13
4.6 Problème rencontré.....	14
5. Les vulnérabilités du protocole ARP.....	19
5.1 Contexte	19
5.2 Absence d'authentification.....	19
5.3 Mise à jour automatique du cache ARP.....	20
5.4 Communication en clair (non chiffrée)	20
5.5 Stateless protocol (sans état).....	20
6. Principales attaques exploitant ARP.....	20
6.1 ARP Spoofing (ou ARP Poisoning)	20
6.2 Attaque Man-In-The-Middle (MITM).....	21
6.3 ARP Flooding.....	21
6.4 Session Hijacking.....	21
7. Mesures de protection	22
7.1 Entrées ARP statiques.....	22
7.2 Utilisation de VLANs	22
7.3 Sécurisation via protocoles supérieurs.....	22
7.4 Exploitation des vulnérabilités ARP avec Kali Linux.....	23
Conclusion.....	27

